

Introdução à Complexidade Computacional - Parte 2

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Relembrando ...

- A **ordem de crescimento** do tempo de execução de um algoritmo dá uma caracterização simples eficiência do algoritmo
⇒ Nos permite comparar o desempenho relativo de algoritmos
 - **Ordenação por intercalação**, no pior caso, é $\Theta(n \lg n)$
 - **Ordenação por inserção**, no pior caso, é $\Theta(n^2)$
 - Ordenação por intercalação é melhor que ordenação por inserção (entrada n grande)
 - Tão logo o tamanho da entrada n se torne suficientemente grande, a ordenação por intercalação, com seu tempo de execução do pior caso $\Theta(n \lg n)$, vence a ordenação por inserção
- Poderíamos determinar o tempo exato de execução do algoritmo
 - Não vale a pena
 - Para entradas grandes o suficiente, as constantes multiplicativas e termos de baixa ordem serão dominados

Crescimento de Funções

- Quando observamos tamanhos de entrada suficientemente grandes para tornar relevante apenas a **ordem de crescimento** do tempo de execução, estamos estudando a **eficiência assintótica** dos algoritmos
 - Estamos preocupados com o modo como o tempo de execução de um algoritmo aumenta com o tamanho da entrada **no limite**
 - À medida que o tamanho da entrada aumenta sem limitação
- Em geral, um algoritmo que é assintoticamente mais eficiente será a melhor escolha para todas as entradas, exceto as muito pequenas

Notação Θ

- Vimos que o tempo de execução do pior caso da ordenação por inserção é $T(n) = \Theta(n^2)$

- **Notação assintótica**

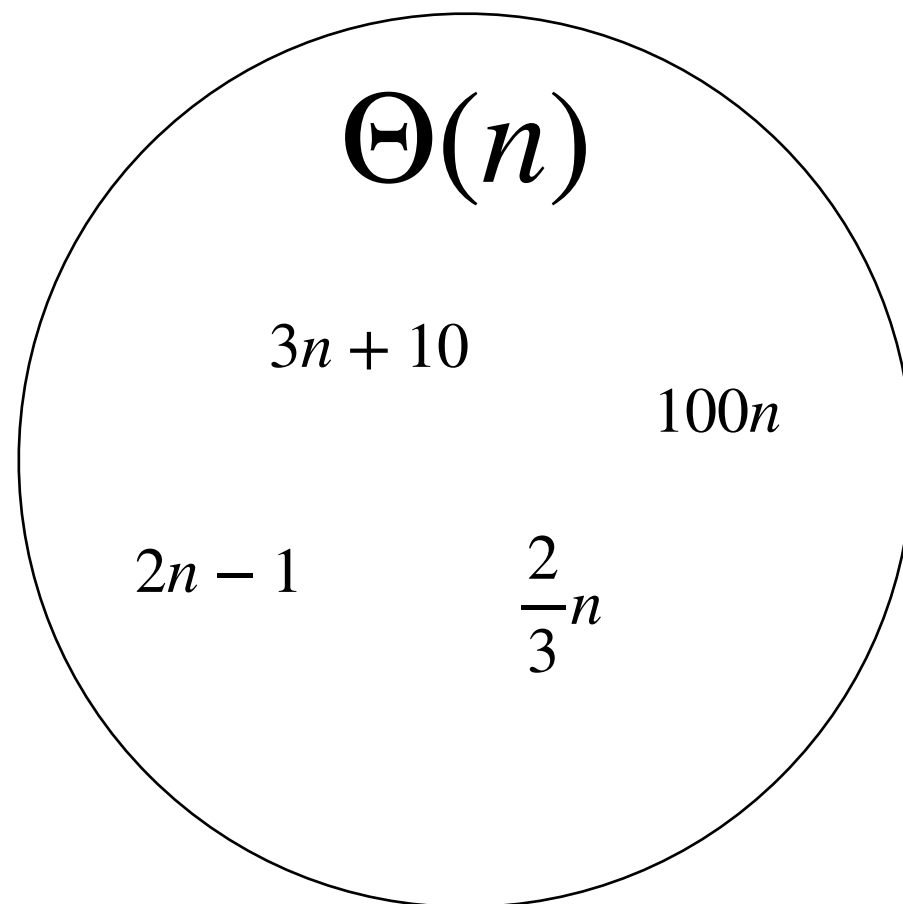
- Mas o que significa essa notação?

- Para uma dada função $g(n)$, denotamos por $\Theta(g(n))$ o conjunto de funções:

$$\Theta(g(n)) = \{f(n) : \text{existem constantes positivas } c_1, c_2 \text{ e } n_0 \text{ tal que } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ para todo } n \geq n_0\}$$

- Uma função $f(n)$ pertence ao conjunto $\Theta(g(n))$ se existirem constantes positivas c_1 e c_2 tais que ela possa ser “encaixada” entre $c_1 g(n)$ e $c_2 g(n)$, para um valor de n suficientemente grande

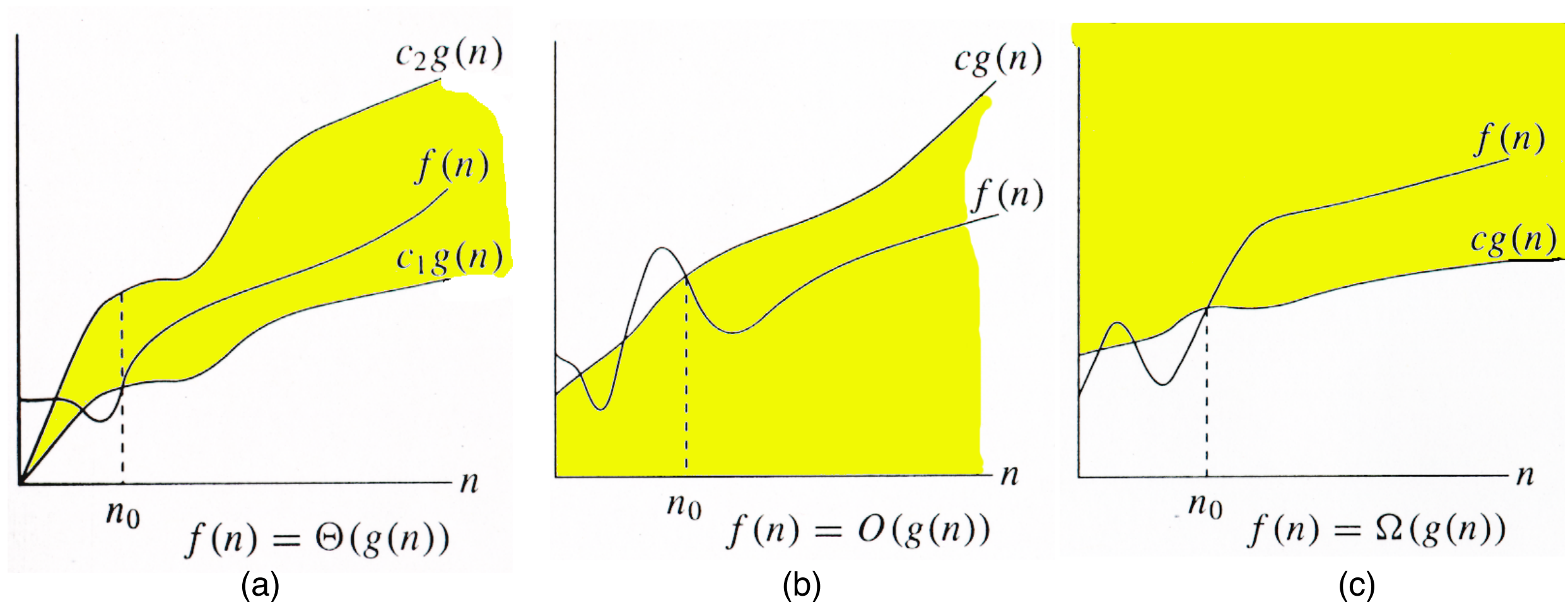
Conjunto de Funções $\Theta(n)$



Notação Θ (Cont.)

- Como $\Theta(g(n))$ é um conjunto, poderíamos escrever “ $f(n) \in \Theta(g(n))$ ” para indicar que $f(n)$ é um membro de (ou pertence a) $\Theta(g(n))$
- Em vez disso, em geral escrevemos “ $f(n) = \Theta(g(n))$ ” para expressar a mesma noção
 - Este abuso da igualdade para denotar pertencimento ao conjunto pode parecer confuso, porém tem suas vantagens

Gráficos das Notações Θ , O e Ω



Exemplos gráficos das notações Θ , O e Ω . Imagem intuitiva das funções $f(n)$ e $g(n)$, onde nós temos que $f(n) = \Theta(g(n))$. Em cada parte, o valor de n_0 é o mínimo valor possível; qualquer valor maior também funcionaria.

Assintoticamente Restrito

- Seja $f(n) = \Theta(g(n))$
 - Para todos os valores de n a direita de n_0 , o valor de $f(n)$ está em ou acima de $c_1g(n)$ e em ou abaixo de $c_2g(n)$
 - Em outras palavras, para todo $n \geq n_0$, a função $f(n)$ é igual a $g(n)$ dentro de um fator constante
 - Dizemos que $g(n)$ é um **limite assintoticamente restrito** para $f(n)$
 - $f(n) \in \Theta(g(n)) \Leftrightarrow g(n)$ é um limite restrito para $f(n)$

Limite Assintoticamente

- Limite assintoticamente superior $\rightarrow$ corresponde a $O(g(n))$: $f(n)$ não cresce mais rápido do que $g(n)$
- Limite assintoticamente inferior $\rightarrow$ corresponde a $\Omega(g(n))$: $f(n)$ não cresce mais devagar do que $g(n)$
- Limite assintoticamente restrito $\rightarrow$ corresponde a $\Theta(g(n))$: $f(n)$ cresce “preso” entre múltiplos constantes de $g(n)$

Não Negativo

- A definição de $\Theta(g(n))$ exige que todo membro $f(n) \in \Theta(g(n))$ seja **assintoticamente não negativo**, isto é, que $f(n)$ seja não negativa sempre que n for suficientemente grande
 - $\Rightarrow$ por consequência, a própria função $g(n)$ deve ser assintoticamente não negativa, senão o conjunto $\Theta(g(n))$ é vazio
 - Essa premissa também se mantém para as outras notações assindéticas definidas nesta aula

Notação Θ : Noção Informal

- Introduzimos uma noção informal da notação Θ que consistia em:
 - Descartar os termos de ordem mais baixa e ignorar o coeficiente inicial do termo de ordem mais alta
- Vamos justificar brevemente essa intuição

Notação Θ : Definição Formal

- Mostraremos que $\frac{1}{2}n^2 - 3n = \Theta(n^2)$
- Para isso, devemos definir constantes positivas c_1 , c_2 e n_0 tais que $c_1n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2n^2$, para todo $n \geq n_0$
- Dividindo por n^2 temos $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$.
 - A desigualdade do lado direito pode ser considerada válida para qualquer valor de $n \geq 1$, escolhendo-se $c_2 \geq \frac{1}{2}$
 - A desigualdade da esquerda pode ser considerada válida para qualquer valor de $n \geq 7$, escolhendo-se $c_1 \leq \frac{1}{14}$
- Assim, escolhendo-se $c_1 = \frac{1}{14}$, $c_2 = \frac{1}{2}$ e $n_0 = 7$, podemos verificar que $\frac{1}{2}n^2 - 3n = \Theta(n^2)$

Notação Θ : Definição Formal (Cont.)

- Certamente, existem outras opções para as constantes, mas o fato importante é que existe **alguma** opção
- Observe que essas constantes dependem da função $\frac{1}{2}n^2 - 3n$
- Uma função diferente pertencente a $\Theta(n^2)$ normalmente exigiria constantes distintas

Notação Θ : Verifique que $6n^3 \neq \Theta(n^2)$

- Mostre que $6n^3 \neq \Theta(n^2)$
 - Suponha, por contradição, que c_2 e n_0 exista, tal que $6n^3 \leq c_2 n^2$ para todo $n \geq n_0$
$$6n^3 \leq c_2 n^2$$
$$n \leq c_2/6 \quad (\text{Divida por } 6n^2)$$
 - Mas então $n \leq c_2/6$, o que não pode ser válido para um valor de n arbitrariamente grande, já que c_2 é constante

Notação Θ : Heurística para Escolher c_1, c_2

- Intuitivamente, termos de baixa ordem de uma função assintótica positiva podem ser ignorados
- Uma pequena fração de um termo de alta ordem é suficiente para os termos de baixa ordem
- Assim, escolha:
 - Ajustar $c_1 <$ coeficiente do termo dominante
 - Ajustar $c_2 >$ coeficiente do termo dominante
 - Isto permite que a desigualdade da definição da notação Θ seja satisfeita

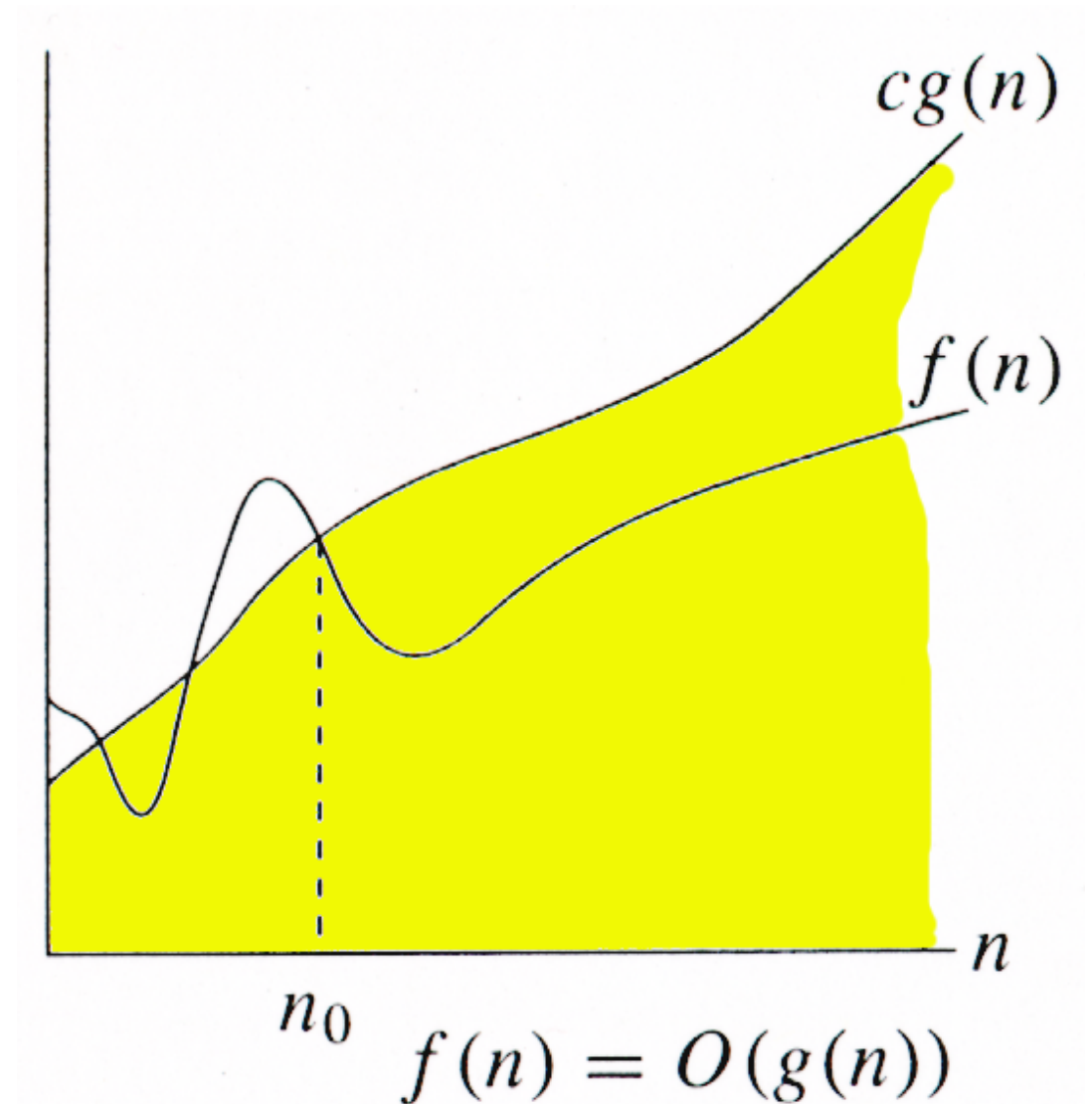
Notação Θ : Função Quadrática

- Considere qualquer função quadrática $f(n) = an^2 + bn + c$, onde a , b e c são constantes e $a > 0$
- Descartando todos os termos de mais baixa ordem e ignorando a constante, obtemos $f(n) = \Theta(n^2)$

Notação O

- Quando temos apenas um **limite assintótico superior**, usamos a notação O

$O(g(n)) = \{f(n) : \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que } 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}$



$$\Theta(g(n)) \subseteq O(n)$$

- Escrevemos $f(n) = O(g(n))$ para indicar que a função $f(n)$ é um membro do conjunto $O(g(n))$
- Observe que $f(n) = \Theta(g(n))$ implica $f(n) = O(g(n))$, já que a notação Θ é uma noção mais forte que a notação O
- Em termos de teoria de conjuntos, escrevemos $\Theta(g(n)) \subseteq O(g(n))$

Limites Justos e Não Justos

- Assim, quando mostramos que qualquer função quadrática $an^2 + bn + c$, onde $a > 0$ está em $\Theta(n^2)$ também mostra que qualquer função quadrática desse tipo está em $O(n^2)$
- Quando $a > 0$, qualquer função linear $an + b$ é $O(n^2)$
- Porém $an + b \neq \Theta(n^2)$
- Na literatura, a notação O é muitas vezes utilizada informalmente para descrever **limites assintóticos “justos”**
 - **Limites muito frouxos** (como dizer que n é $O(n^3)$) são verdadeiros, mas não transmitem informação útil
 - **Limites justos** (do tipo “tamanho certo da ordem”) é o que geralmente interessa em análise de algoritmos

Estrutura Global do Algoritmo

- Usando a notação O , podemos descrever frequentemente o **tempo de execução de um algoritmo** apenas inspecionando a estrutura global do algoritmo
- Por exemplo, a estrutura de laço duplamente aninhado do algoritmo de ordenação por inserção produz imediatamente um **limite superior** de $O(n^2)$ para o tempo de execução do **pior caso**
 - O limite $O(n^2)$ se aplica a seu tempo de execução para toda entrada
 - Por sua vez, limite $\Theta(n^2)$ para o **pior caso**, não implica um limite $\Theta(n^2)$ para toda entrada

Notação O: Mostre que

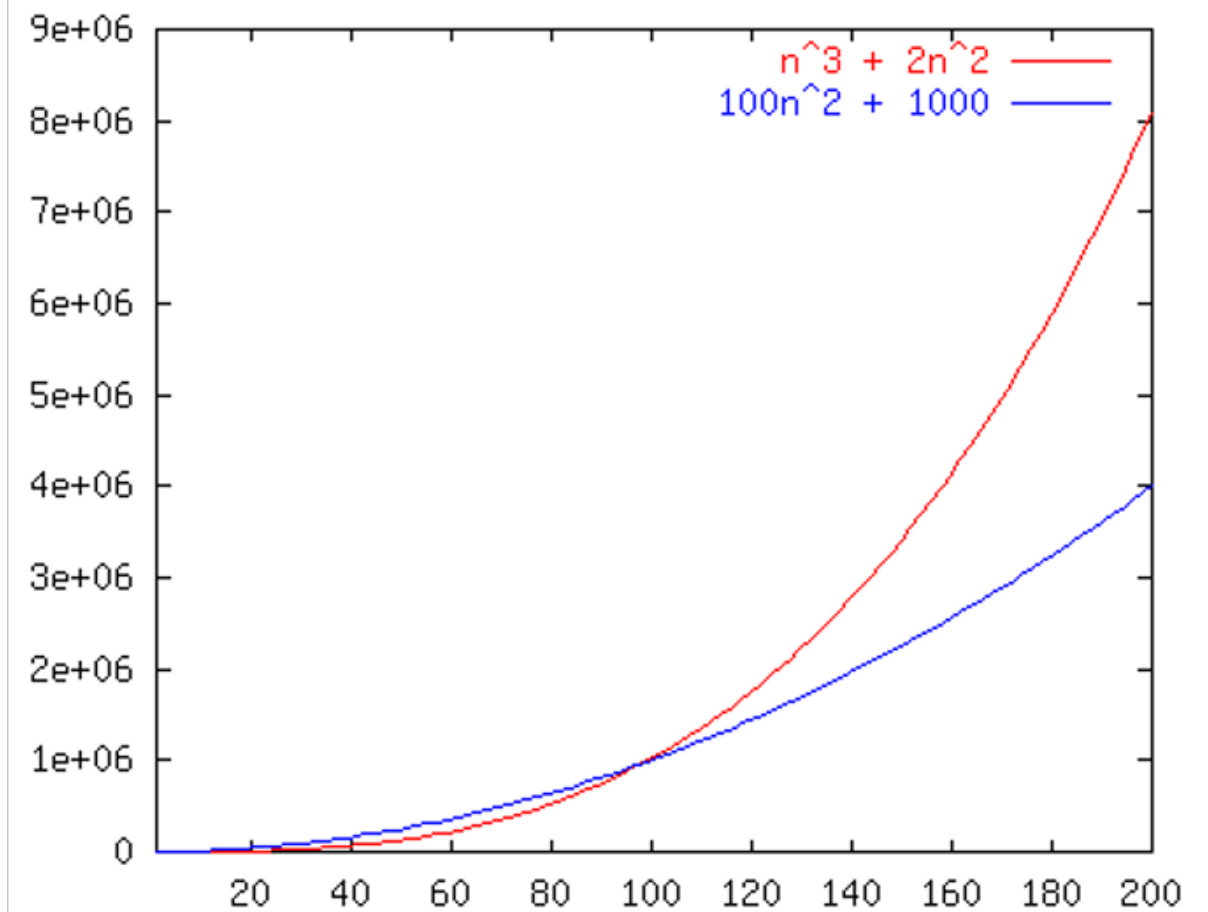
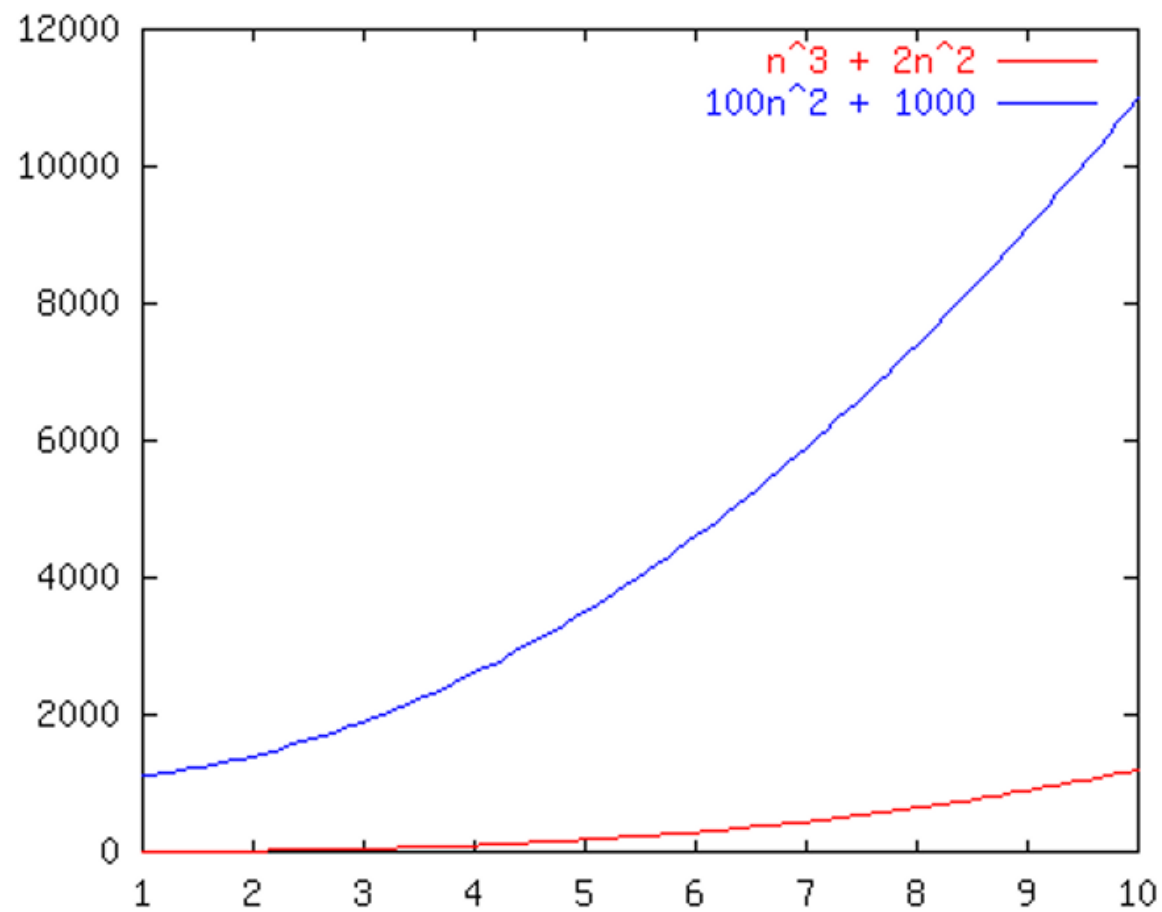
- Mostre que $3n^3 = O(n^4)$
- Pela definição, precisamos de constantes $c > 0$ e $n_0 \geq 1$ tais que
 $3n^3 \leq cn^4$ para todo $n \geq n_0$
Dividindo por $n^3 > 0$:
 $3 \leq cn$.
Tomando $c = 4$, obtemos a condição
 $n \geq 3/4$.
Assim, basta escolher $n_0 = 1$. Logo, para todo $n \geq 1$:
 $3n^3 \leq 4n^4$.
Portanto, $3n^3 \in O(n^4)$. $\square$

Notação O: Exemplos

- $f(n) = n^2 - 1 \Rightarrow f(n) = O(n^2)$.
- $f(n) = n^2 - 1 \Rightarrow f(n) = O(n^3)$.
- $f(n) = 403 \Rightarrow f(n) = O(1)$.
- $f(n) = 5 + 2 \log n + 3 \log^2 n \Rightarrow f(n) = O(\log^2 n)$.
- $f(n) = 5 + 2 \log n + 3 \log^2 n \Rightarrow f(n) = O(n)$.
- $f(n) = 3n + 5 \log n + 2 \Rightarrow f(n) = O(n)$.
- $f(n) = 5 \cdot 2^n + 5n^{10} \Rightarrow f(n) = O(2^n)$.

Corrida 1

$n^3 + 2n^2$ vs. $100n^2 + 1000$

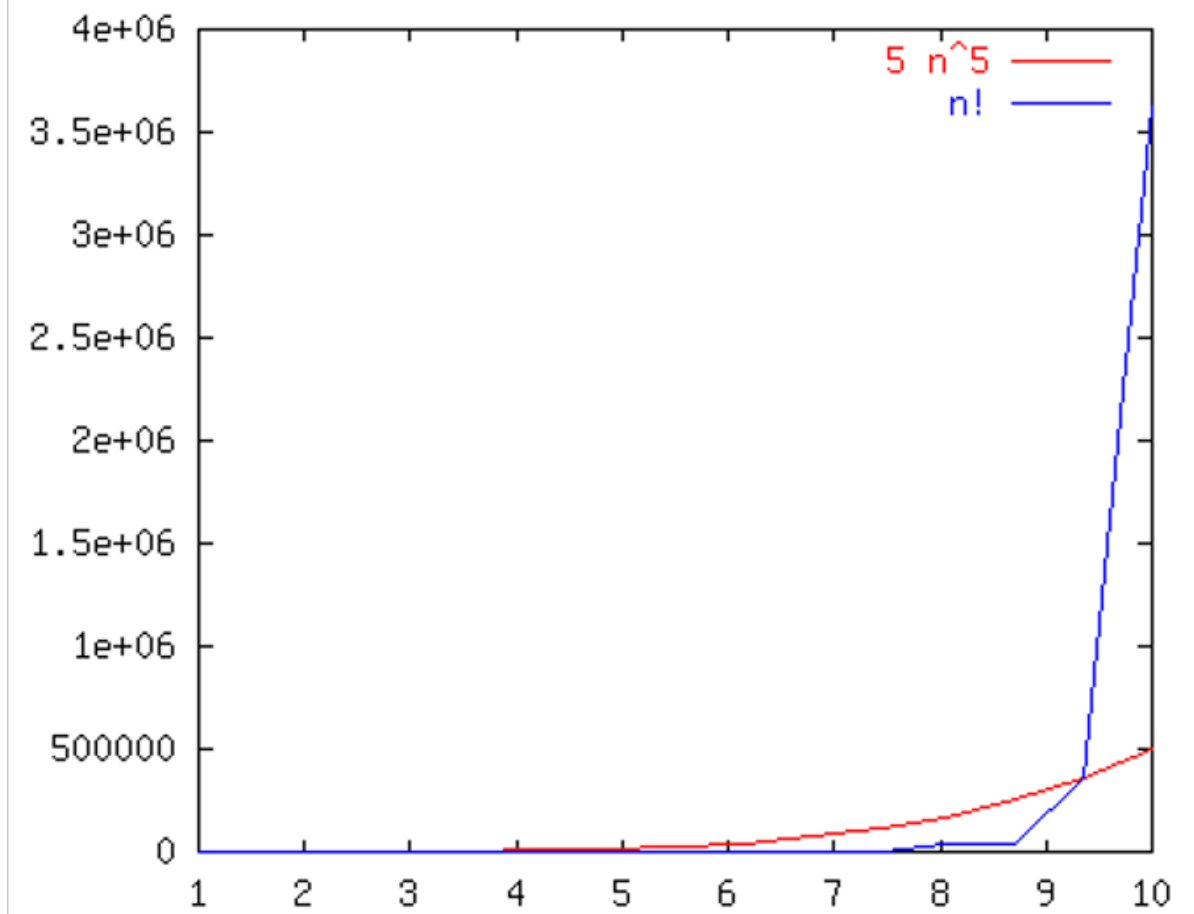
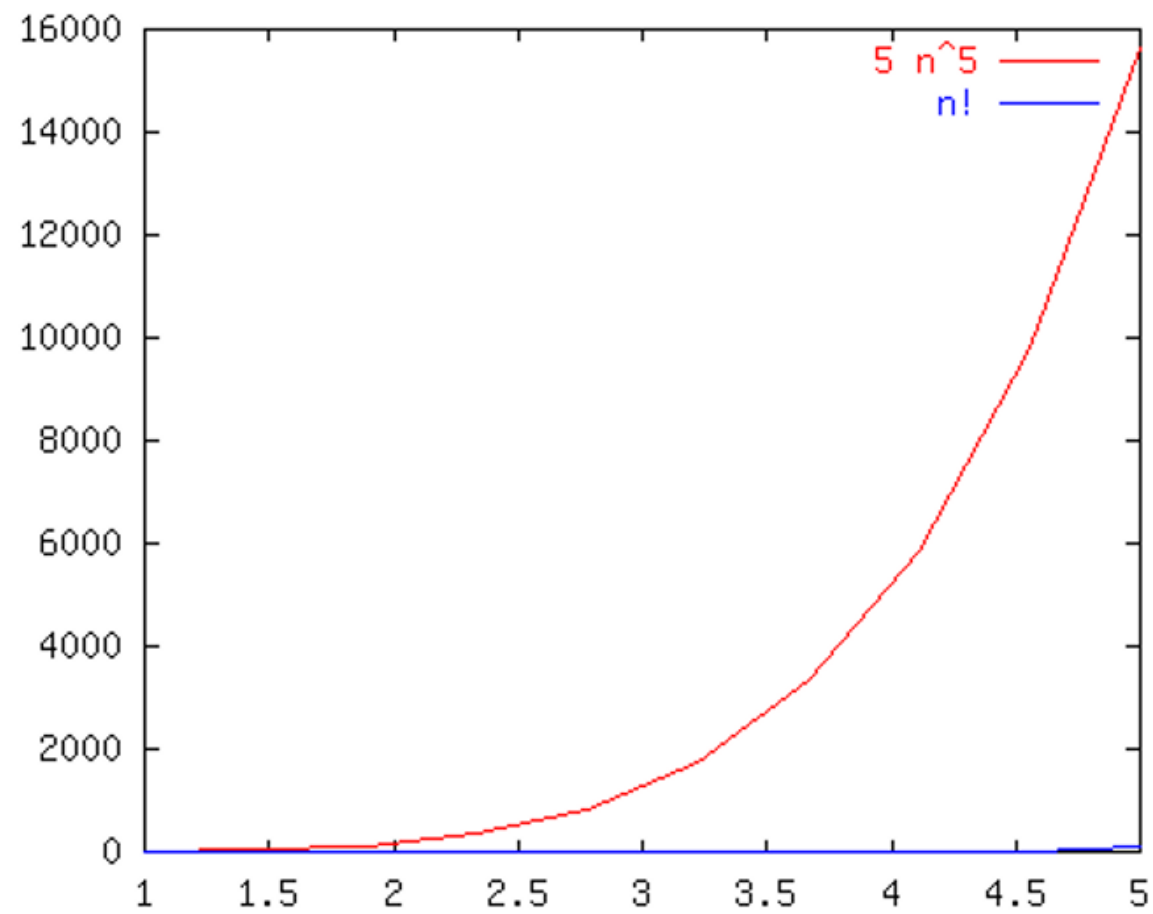


Corrida 2

$5n^5$

VS.

$n!$

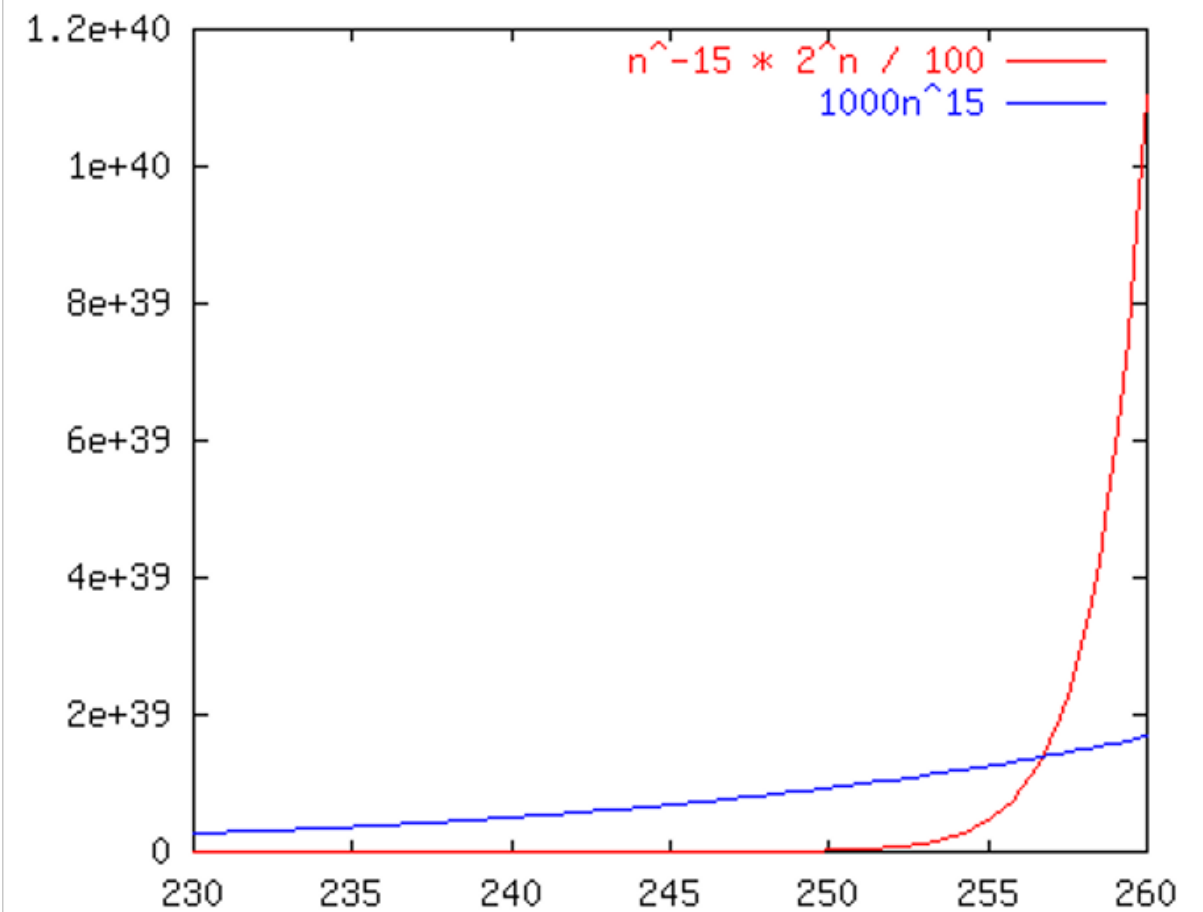
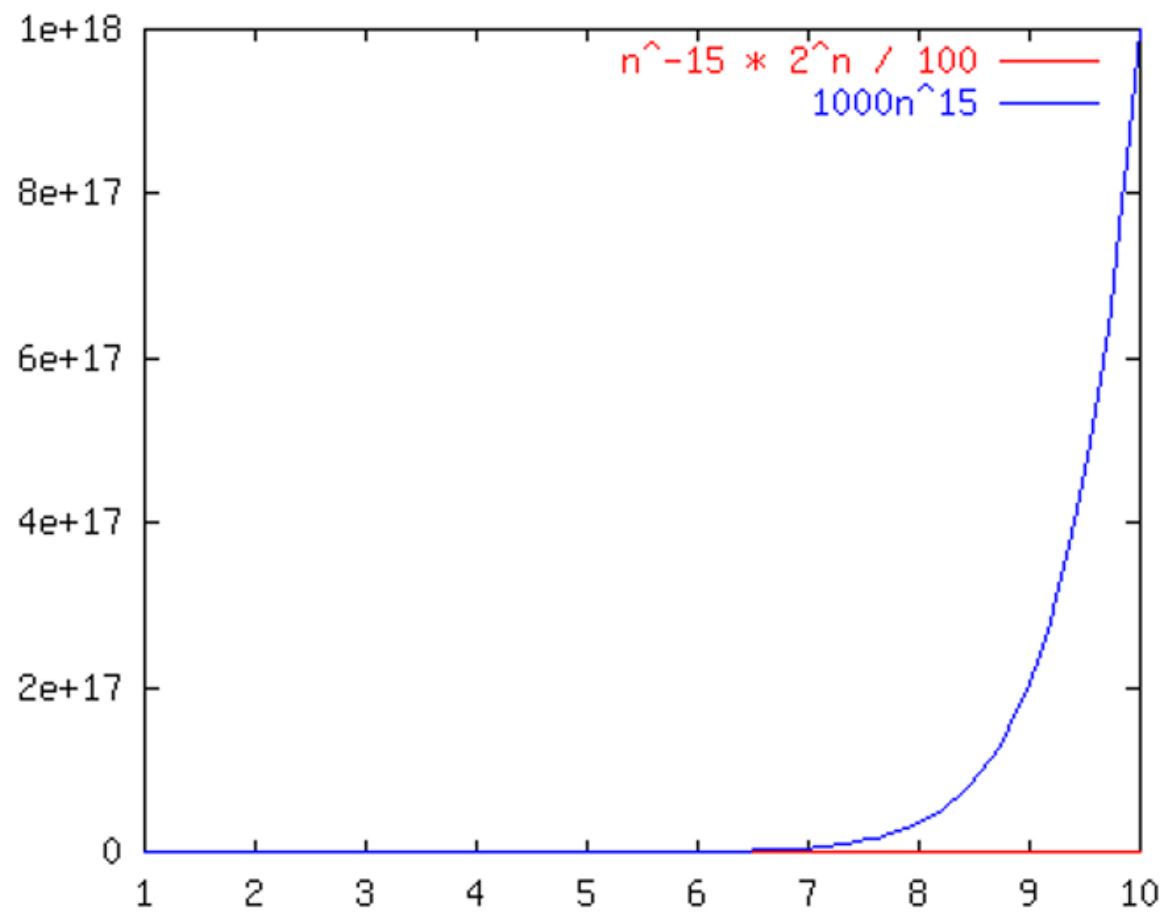


Corrida 3

$$n^{-15} 2^n / 100$$

vs.

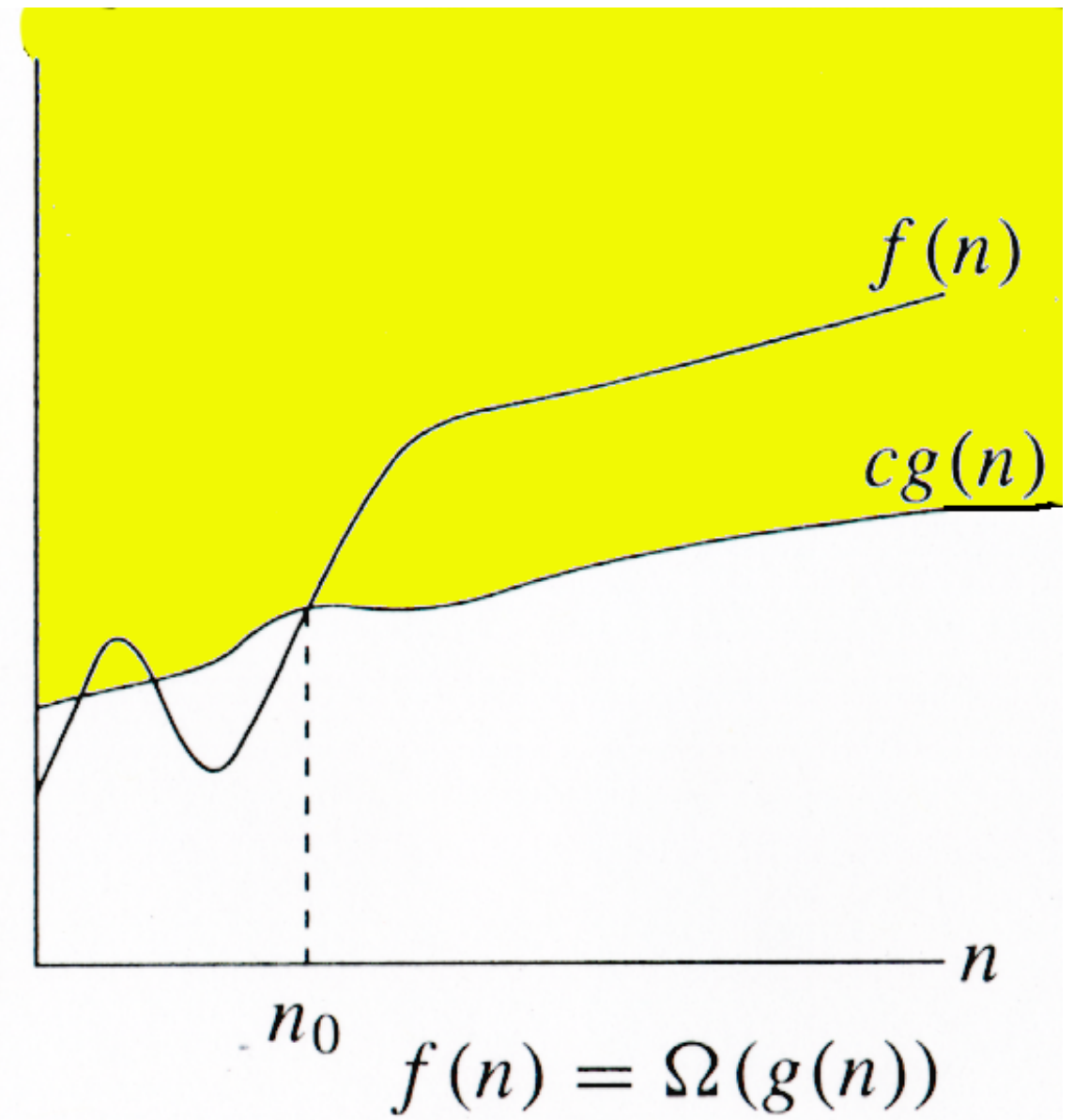
$$1000n^{15}$$



Notação Ω

- Quando temos apenas um **limite assintótico inferior**, usamos a notação Ω (ômega)

$\Omega(g(n)) = \{f(n) : \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que } 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}$



Notação Ω (Cont.)

- Por exemplo, se $f(n) = n^2 - 1$, então
 - são válidas as igualdades $f = \Omega(n^2)$, $f = \Omega(n)$ e $f = \Omega(1)$
 - mas não $f(n) = \Omega(n^3)$
- Uma vez que a notação Ω descreve o limite inferior, quando nós utilizamos para caracterizar o tempo de execução do **melhor caso**, por consequência nós limitamos o tempo de execução do algoritmo para qualquer entrada
 - Por exemplo, o **melhor caso** da ordenação por inserção é $\Omega(n)$
 - Isto implica que o tempo de execução da ordenação por inserção é $\Omega(n)$

Teorema

- **Teorema**

Para quaisquer duas funções $f(n)$ e $g(n)$, temos
 $f(n) = \Theta(g(n))$ se e somente se $f(n) = O(g(n))$ e
 $f(n) = \Omega(g(n))$

- Vimos anteriormente que $an^2 + bn + c = \Theta(n^2)$, para quaisquer constantes a, b e c , onde $a > 0$
- Isto implica imediatamente que:
 $\Rightarrow an^2 + bn + c = \Omega(n^2)$
 $\Rightarrow an^2 + bn + c = O(n^2)$

Notação Ω : Mostre que

- Mostre que $n^2 - 1 = \Omega(n^2)$
- Pela definição, precisamos de constantes $c > 0$ e n_0 tais que $n^2 - 1 \geq cn^2$ para todo $n \geq n_0$.

Tomemos $c = \frac{1}{2}$. Então

$$n^2 - 1 \geq \frac{1}{2}n^2 \Leftrightarrow n^2 \left(1 - \frac{1}{2}\right) \geq 1 \Leftrightarrow \frac{1}{2}n^2 \geq 1$$
$$\Leftrightarrow n \geq \sqrt{2}.$$

Como trabalhamos com n natural, basta escolher $n_0 = 2$.

Logo, para todo $n \geq 2$,

$$n^2 - 1 \geq \frac{1}{2}n^2$$

o que prova $n^2 - 1 \in \Omega(n^2)$. $\square$

Analogia Comparação Assintótica

- Podemos traçar uma analogia entre a comparação assintótica de duas funções f e g e a comparação de dois números reais a e b :

$$f(n) = O(g(n)) \approx a \leq b,$$

$$f(n) = \Omega(g(n)) \approx a \geq b,$$

$$f(n) = \Theta(g(n)) \approx a = b.$$

Problemas Intratáveis

- Suponha que uma máquina moderna pode realizar cerca de 10^{10} operações por segundo
- Suponha uma entrada $n = 100$ para um algoritmo de complexidade 2^n
 - São aproximadamente 10^{30} operações
- Leva-se 10^{20} segundos
- Um dia tem 10^5 segundos
 - Isso significa 10^{15} dias. Ou aproximadamente 10^{13} anos
- A idade do universo foi calculada em cerca de 10^{10} anos ...

Complexidade Algoritmos de Ordenação

Name ↕	Best ↕	Average ↕	Worst ↕	Memory ↕	Stable ↕	Method ↕	Other notes ↕
Quicksort	$n \log n$ variation is n	$n \log n$	n^2	$\log n$ on average, worst case is n ; Sedgewick variation is $\log n$ worst case	typical in-place sort is not stable; stable versions exist	Partitioning	Quicksort is usually done in place with $O(\log n)$ stack space. ^{[2][3]}
Merge sort	$n \log n$	$n \log n$	$n \log n$	n	Yes	Merging	Highly parallelizable (up to $O(\log n)$ using the Three Hungarians' Algorithm ^[4] or, more practically, Cole's parallel merge sort) for processing large amounts of data.
In-place merge sort	—	—	$n \log^2 n$	1	Yes	Merging	Can be implemented as a stable sort based on stable in-place merging. ^[5]
Heapsort	$n \log n$	$n \log n$	$n \log n$	1	No	Selection	
Insertion sort	n	n^2	n^2	1	Yes	Insertion	$O(n + d)$, in the worst case over sequences that have d inversions.
Introsort	$n \log n$	$n \log n$	$n \log n$	$\log n$	No	Partitioning & Selection	Used in several STL implementations.
Selection sort	n^2	n^2	n^2	1	No	Selection	Stable with $O(n)$ extra space, for example using lists. ^[6]
Timsort	n	$n \log n$	$n \log n$	n	Yes	Insertion & Merging	Makes n comparisons when the data is already sorted or reverse sorted.
Cubesort	n	$n \log n$	$n \log n$	n	Yes	Insertion	Makes n comparisons when the data is already sorted or reverse sorted.
Shell sort	n	$n \log^2 n$ or $n^{3/2}$	Depends on gap sequence; best known is $n \log^2 n$	1	No	Insertion	Small code size, no use of call stack, reasonably fast, useful where memory is at a premium such as embedded and older mainframe applications.

Referências

- CORMEN, T. H.[et al]. Algoritmos: teoria e prática. 3^a ed. Rio de Janeiro: Elsevier, 2012.
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de dados e seus algoritmos. 3. ed. Rio de Janeiro, RJ: LTC, 2010. 302p.